

Software Engineering

Dr. Md. Nasim Adnan

Some Software Failures: Disney's Lion King, 1994–1995

- In the fall of 1994, the Disney company released its first multimedia CD-ROM game for children, The Lion King Animated Storybook.
- Although many other companies had been marketing children's programs for years, this was Disney's first venture into the market and it was highly promoted and advertised. Sales were huge. It was "the game to buy" for children that holiday season.
- What happened, however, was a huge debacle. On December 26, the day after Christmas, Disney's customer support phones began to ring, and ring, and ring.
- Soon the phone support technicians were swamped with calls from angry parents with crying children who couldn't get the software to work. Numerous stories appeared in newspapers and on TV news.

NASA Mars Polar Lander, 1999

- On December 3, 1999, NASA's Mars Polar Lander disappeared during its landing attempt on the Martian surface. A Failure Review Board investigated the failure and determined that the most likely reason for the malfunction was the unexpected setting of a single data bit.
- Most alarming was why the problem wasn't caught by internal tests. In theory, the plan for landing was this: As the lander fell to the surface, it was to deploy a parachute to slow its descent.
- A few seconds after the chute deployed, the probe's three legs were to snap open and latch into position for landing. When the probe was about 1,800 meters from the surface, it was to release the parachute and ignite its landing thrusters to gently lower it the remaining distance to the ground.

NASA Mars Polar Lander, 1999

- Unfortunately, the Failure Review Board discovered in their tests that in most cases when the legs snapped open for landing, a mechanical vibration also tripped the touch-down switch, setting the fatal bit. It's very probable that, thinking it had landed, the computer turned off the thrusters and the Mars Polar Lander smashed to pieces after falling 1,800 meters to the surface.

The USA Star-Wars Program

- 'Patriot missile' was the result of the USA 'Star Wars' program. This missile was used for the first time in the Gulf war against the Scud missile of Iraq. Surprisingly, 'Patriot missiles' failed many times to hit the targeted Scud missile. One of the failures killed 28 American soldiers in Dhahran, Saudi Arabia.
- An investigation team was constituted to identify the cause of failure. The team re-looked at every dimension of the product and found the reason for the failure. The cause of the failure was a software fault. There was a slight timing error in the system's clock after 14 hours of its operation.
- Hence, the tracking system was not accurate after 14 hours of operations and at the time of the Dhahran attack, the system had already operated for more than 100 hours.

The Y2K (Year 2000) Bug, circa 1974

- Sometime in the early 1970s a computer programmer—let's suppose his name was Dave—was working on a payroll system for his company. The computer he was using had very little memory for storage, forcing him to conserve every last byte he could. Dave was proud that he could pack his programs more tightly than any of his peers.
- One method he used was to shorten dates from their 4-digit format, such as 1973, to a 2-digit format, such as 73. Because his payroll program relied heavily on date processing, Dave could save lots of expensive memory space. He briefly considered the problems that might occur when the current year hit 2000 and his program began doing computations on years such as 00 and 01.

The Y2K (Year 2000) Bug, circa 1974

- He knew there would be problems but decided that his program would surely be replaced or updated in 25 years and his immediate tasks were more important than planning for something that far out in time. After all, he had a deadline to meet. In 1995, Dave's program was still being used, Dave was retired, and no one knew how to get into the program to check if it was Y2K compliant, let alone how to fix it. It's estimated that several hundred billion dollars were spent, worldwide, to replace or update computer programs such as Dave's, to fix potential Year 2000 failures.

Intel Pentium Floating-Point Division Bug, 1994

- Enter the following equation into your PC's calculator: $(4195835 / 3145727) * 3145727 - 4195835$
- If the answer is zero, your computer is just fine. If you get anything else, you have an old Intel Pentium CPU with a floating-point division bug—a software bug burned into a computer chip and reproduced over and over in the manufacturing process.
- On October 30, 1994, Dr. Thomas R. Nicely of Lynchburg (Virginia) College traced an unexpected result from one of his experiments to an incorrect answer by a division problem solved on his Pentium PC.

- He posted his find on the Internet and soon afterward a firestorm erupted as numerous other people duplicated his problem and found additional situations that resulted in wrong answers.
- Fortunately, these cases were rare and resulted in wrong answers only for extremely math-intensive, scientific, and engineering calculations. Most people would never encounter them doing their taxes or running their businesses.

- What makes this story notable isn't the bug, but the way Intel handled the situation:
 - Their software test engineers had found the problem while performing their own tests before the chip was released. Intel's management decided that the problem wasn't severe enough or likely enough to warrant fixing it, or even publicizing it.
 - Once the bug was found, Intel attempted to diminish its perceived severity through press releases and public statements.
 - When pressured, Intel offered to replace the faulty chips, but only if a user could prove that he was affected by the bug

- There was a public outcry. Internet newsgroups were jammed with irate customers demanding that Intel fix the problem. News stories painted the company as uncaring and incredulous.
- In the end, Intel apologized for the way it handled the bug and took a charge of over \$400 million to cover the costs of replacing bad chips. Intel now reports known problems on its Web site and carefully monitors customer feedback on Internet newsgroups

The Explosion of the Ariane 5 Rocket

- The Ariane 5 rocket was designed by European Space Agency and it was launched on June 4, 1996. It was an unmanned rocket and unfortunately exploded only after 40 seconds of its take off from Kourou, French Guiana. The design and development took ten long years with a cost of \$7 billion.
- An enquiry board was constituted to find the reasons of the explosion. The board identified the cause and mentioned in its report that : “The failure of the Ariane 5 was caused by the complete loss of guidance and altitude information, 37 seconds after start of the main engine ignition sequence (30 seconds after lift-off).

- Unfortunately, the navigation system of Ariane 4 was used in Ariane 5 without proper testing and analysis. The Ariane 5 was a high speed rocket with higher value of an internal alignment function, known as horizontal bias.
- This value is for the calculation of horizontal velocity. On the day of the explosion, this value was more than expectations due to different trajectory of this rocket as compared to the Ariane 4.

Software Testing

- Testing is a set of activities that can be planned in advance and conducted systematically.
- Since developers create the software, they usually know it best.

Organizing for Software Testing

- In every software project, a natural conflict happens when testing starts. The people who created the software are now asked to test it. At first, this might seem fine—after all, no one knows the program better than its developers.
- However, these same developers often have a personal interest in proving that the program has no errors, works according to customer requirements, and will be finished on time and within budget. These interests can make them less likely to perform thorough testing.

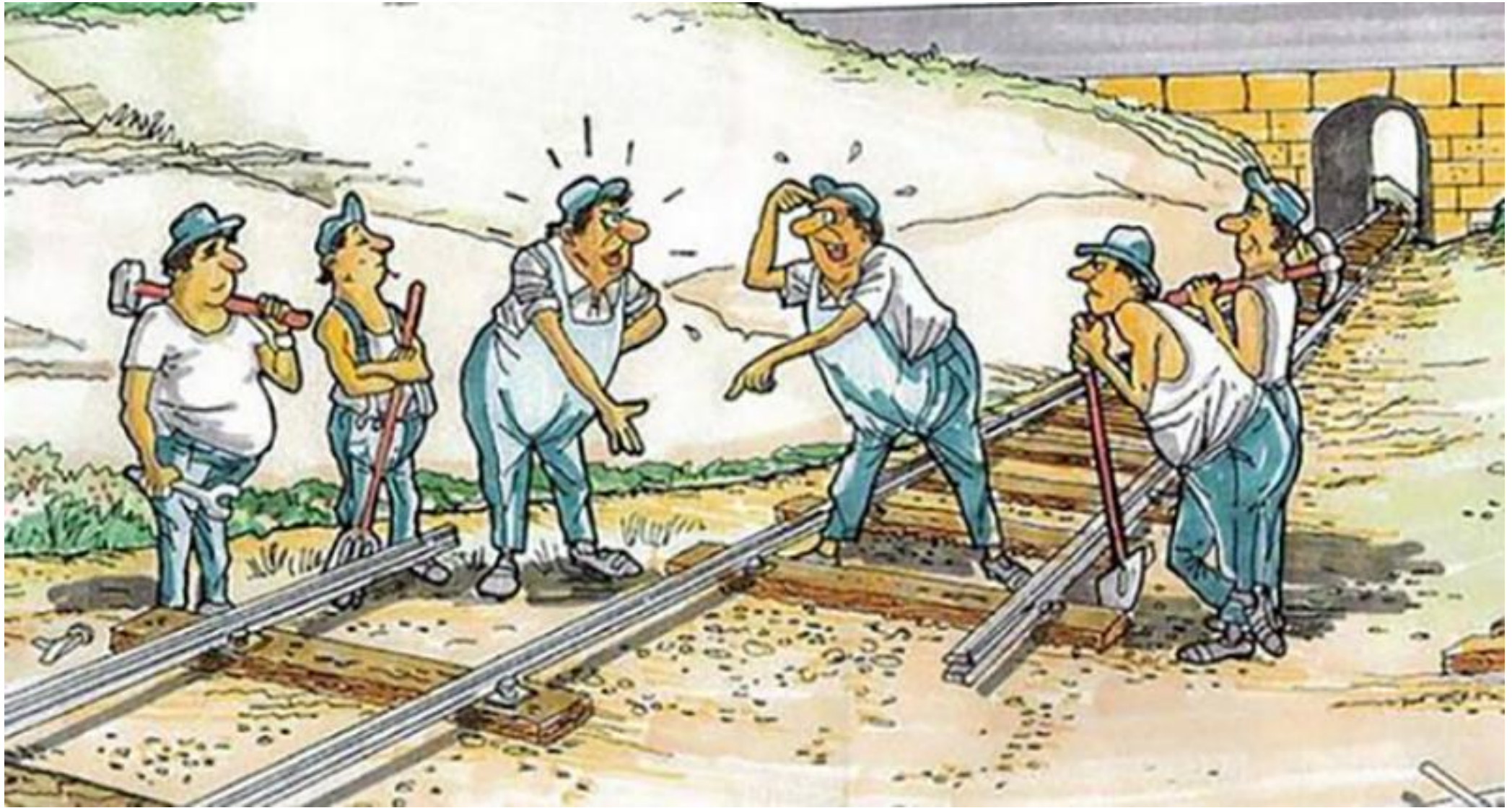
- From a psychological point of view, software analysis, design, and coding are creative activities. A team of software engineers studies the requirements, creates models, and then develops a computer program along with its documentation.
- Like any builder, software engineers feel proud of what they have created and may not welcome anyone trying to find faults in it. When testing begins, it involves trying to 'break' the program the engineers have built. To the builder, this can feel like a destructive activity.
- Because of this, the builder may test the program gently, focusing on proving it works instead of actively looking for errors.
- Unfortunately, errors will still exist and if the software engineers don't find them, the customers will.

There are some common misunderstandings that you might get from the earlier discussion:

1. That the software developer should not do any testing at all.
2. That the software should simply be “thrown over the wall” to other people who will test it harshly.
3. That testers only join the project when testing is about to start.

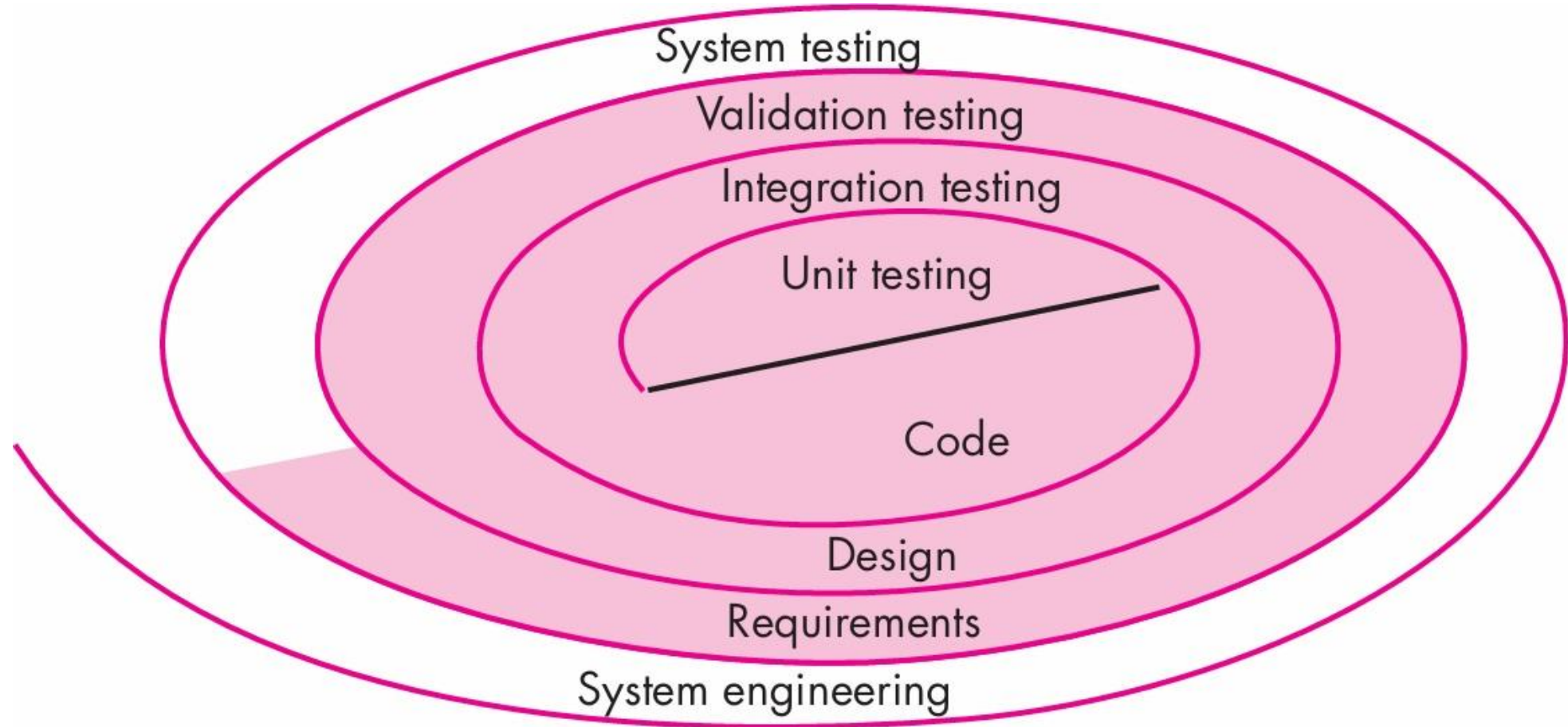
All of these ideas are wrong.

The software developer is always responsible for testing the individual units (components) of the program to make sure each one works as intended. In many cases, the developer also performs integration testing, which is the step where the different parts are combined to build the complete software system.



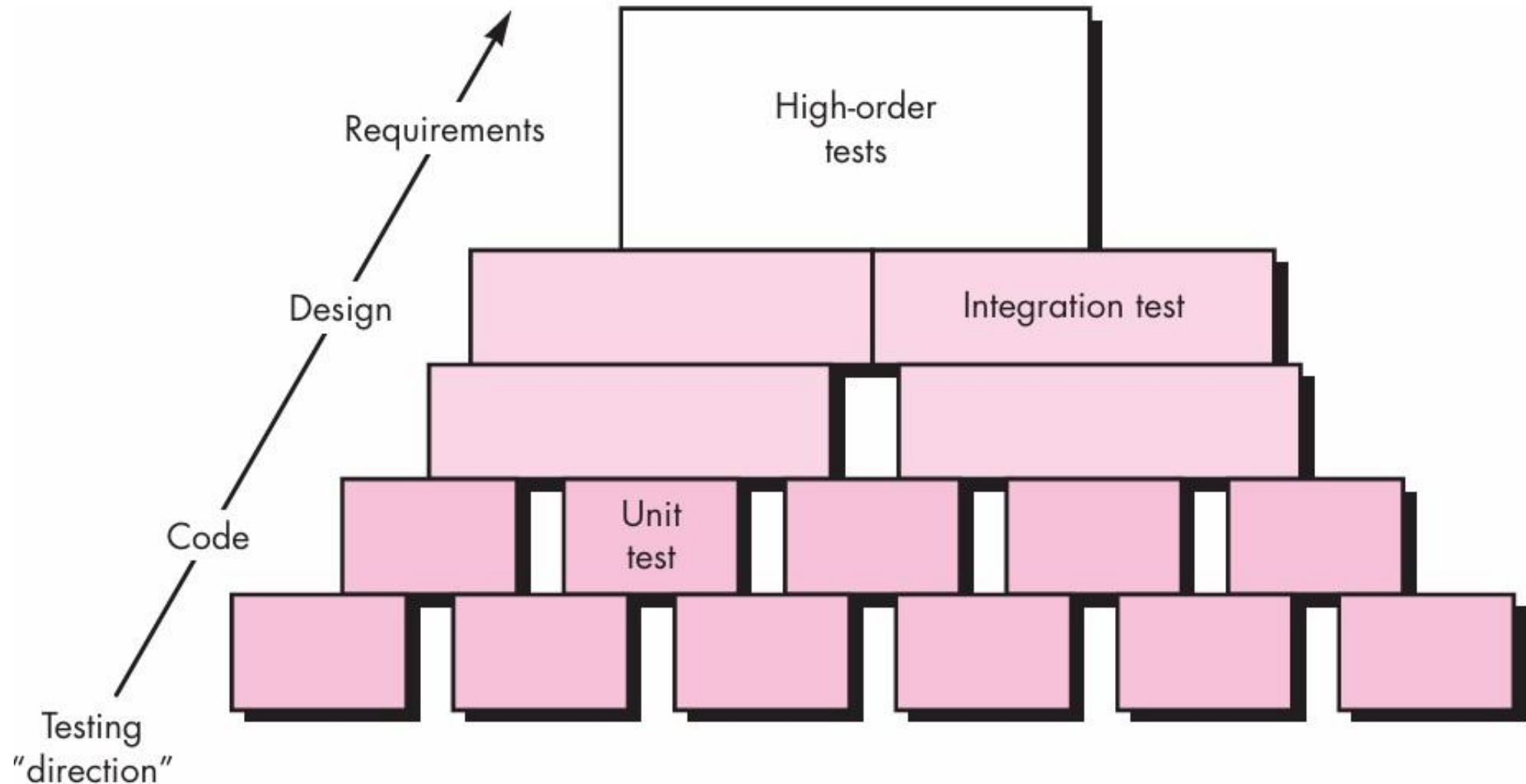
- An Independent Test Group (ITG) gets involved only after the software architecture is finished.
- The job of the ITG is to avoid the problems that come from letting the builder test their own work. Independent testing removes the conflict of interest, since ITG members are specifically paid to find errors.
- However, the program is not simply handed over to the ITG and forgotten. The developer and ITG work together throughout the project to make sure testing is thorough. During testing, the developer must be available to fix any errors that the ITG discovers.

Software Testing Strategy—The Big Picture



- A software testing strategy can be understood using the spiral model.
- Unit testing starts at the center (vortex) of the spiral and focuses on testing each small part (unit) of the software's source code.
- As testing moves outward along the spiral, it reaches integration testing, which focuses on testing how the software parts work together in the overall design and architecture.
- Further outward is validation testing, where the software is checked to see if it meets the original requirements set during the requirements modeling phase.
- Finally, at the outer edge of the spiral, system testing happens. This tests the entire software along with other system components as a complete whole.

- From a procedural point of view, testing in software engineering is a series of four steps that are carried out one after another in order.



Test Strategies for Conventional Software

- When to start testing?

There are many strategies that can be used to test software. Testing should ideally start as soon as any part of the system is ready, rather than waiting for the entire system to be completely built. While some might think it is simpler to test the software only after the full system is completed, this approach usually does not work well. Waiting until the end often results in software that contains many bugs, which can disappoint both developers and users. At the other extreme, testing on a daily basis, whenever any part of the system is constructed. This approach, although less appealing to many, can be very effective.

- A testing strategy that is chosen by most software teams falls between the two extremes.
- This strategy uses an incremental approach: it starts by testing individual program units, then moves on to tests that help integrate those units, and finally performs tests on the complete system.
- Each of these types of tests is explained in the following sections.

Unit Testing

- Unit testing checks the smallest part of software design —the software component or module. Using the design description for that component, important control paths are tested to find errors inside the module's limits.
- The unit test looks closely at the internal logic and data structures within the module. This means every independent path through the control flow is tested to make sure all parts of the module have run at least once. Boundary conditions are also tested to confirm the module works correctly at the edges where processing is limited or controlled. Finally, all error-handling paths are checked.
- This type of testing can be conducted in parallel for multiple components.

Unit Testing

- **Most common errors**
 - Misunderstood/incorrect arithmetic precedence
 - Incorrect initializations
 - Incorrect comparison of variables
 - Precision inaccuracy
 - Incorrect symbolic representation of an expression
 - Mixed mode operations
- Unit testing is simplified when a component with high cohesion is designed.

Integration Testing

- A beginner in software might ask a reasonable question after all modules have passed unit tests: “If each module works by itself, why should we worry that they won’t work when put together?”
- The real problem is in “putting them together” —the interfaces between modules.
- Data can be lost when passing through an interface; one component might accidentally cause problems for another; small functions combined might not create the big function expected; small acceptable inaccuracies can add up to big, unacceptable errors; global data shared by many parts can cause trouble. Unfortunately, these problems and many more can happen.

Integration testing is the process of combining and testing modules together to ensure they work as a group. There are different ways to approach integration testing:

Top-Down Integration Testing:

Testing starts from the top-level modules and moves downward to lower-level modules. The higher modules are tested first, and lower-level modules are integrated step by step. If lower modules are not yet ready, “stubs” (simple placeholders) are used to simulate their behavior. This approach helps test major control flows early.

Bottom-Up Integration Testing:

Testing begins with the lowest-level modules first. These are tested individually, then combined step by step to form higher-level modules. If higher modules are not ready yet, “drivers” (test code) simulate them. This approach helps ensure that the fundamental parts work well before building the full system.

Top-Down or Bottom-Up?

Both approaches have advantages and disadvantages. The choice depends on the project needs, resources, and risks. Sometimes teams combine both methods in a “sandwich” or “mixed” approach, testing parts of the system top-down and others bottom-up to gain the benefits of both

- Selection of integration testing strategy depends upon software characteristics and sometimes project schedule.
- Sometimes a combined approach (sandwich testing) is used ...

Cohesion and Coupling

- Cohesion refers to how closely related the responsibilities of a single module or class are. A highly cohesive module performs a single, well-defined task, making it easier to understand, test, and maintain.
- For example, a clsUserManager class that only handles user-related operations (like creating, updating, or deleting users) is cohesive.
 - High cohesion = focused, organized, and reusable code.
 - Low cohesion = scattered responsibilities, harder to maintain.

Cohesion and Coupling

- Coupling describes how dependent one module is on another. Low coupling means modules interact through well-defined interfaces and don't rely heavily on each other's internal workings. This makes the system more flexible and easier to maintain or change
 - Low coupling = Modules can change or improve separately without affecting each other.
 - High coupling = changes in one module may break others.
- The sweet spot in software design is **high cohesion and low coupling.**

Regression testing (Toughest)

- Each time a new module is added, the software changes. New data flow paths are established, new I/O may occur, and new control logic is invoked. These changes may cause problems with functions that previously worked smoothly.
- In the context of an integration test strategy, regression testing is the re-execution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects

Sanity Testing

- A sanity test is a narrow regression test that focuses on one or a few areas of functionality
- Sanity testing is usually narrow and deep.
- A Sanity test is used to determine a small (usually very critical/ Shamsul Alamin Pharma) section of the application (a few areas of functionality) is still working after a minor change.

Smoke testing

- Smoke testing originates in the hardware testing practice of turning on a new piece of hardware for the first time and considering it a success if it does not catch fire and smoke.
- In the software industry, smoke testing is a method where all parts of the application are tested in a broad and general way, without going too deep into any specific area.
- It is scripted, either using a written set of tests or an automated test (selenium)
- Smoke testing is conducted to ensure whether the most important functions of a program are working, but not bothering with finer details.
- Smoke testing is like normal health check up.

Test Strategies for Object-Oriented Software

- The objective of testing is to find out the greatest possible number of errors with a manageable amount of effort applied over realistic time span, remains the same for Object Oriented Software.

Unit Testing in the OO Context

Class testing in object-oriented (OO) software is like unit testing in normal software. In normal software, unit testing checks the details of algorithms and how data moves through a module's interface. In OO software, class testing focuses on the actions (methods) inside the class and how the class's state (its data/attributes) changes

Integration Testing in the OO Context

- There are two different strategies for integration testing of OO systems.
 - Thread-based testing: Test all the classes needed to handle one input or event. Each thread is tested separately.
 - Use-based testing: Start by testing independent classes. Then test dependent classes step by step, until the whole system is complete.
 - Cluster testing: Test a group of related classes together by creating test cases that check how they work with each other.

Validation Testing in an OO Context

- In validation testing, there is no difference between normal software and object-oriented software.
- After validation testing, two outcomes are possible:
- The software matches the requirements → it is accepted.
- The software does not match → a list of problems is made.
- Errors found at this stage are hard to fix before the delivery date.
- In such cases, developers usually discuss and negotiate with customers.

Validation Testing: How?

- Alpha Testing

Alpha testing is a type of software testing done at the developer's site before the product is released to customers. Since it is almost impossible for developers to predict exactly how customers will use the software, this testing is carried out in a natural environment where typical users work with the software. The developer observes their usage "over the shoulder," records any errors or problems, and ensures that all requirements are validated. This process usually takes a relatively longer time because it covers all aspects of the system.

- Beta Testing

Beta testing is performed at the end user's site, where the software is used just like a real, "live" application. It usually takes a bit long time since it also checks whether all requirements are properly met. This testing is carried out by several groups of users, who use the software in their normal environment. Any problems or errors they encounter are reported, collected, and then fixed by the developers before the software's final release.

System Testing

System Testing is the process of testing the complete and fully integrated software system to check if it meets the specified requirements. It tests the entire system as a whole – including software, hardware, and other components – to ensure everything works together correctly before the software is delivered to users.

- Deployment Testing
 - Deployment testing checks if the software works properly in the real environment where it will be used.
- Security testing
 - Security testing checks if the software is safe from threats.
- Performance testing
 - Run-time performance
- Stress testing
 - How high can the software handle before it fails
- Recovery testing
 - Fault tolerant

Overall Test Characteristics

- A good test has a high probability of finding errors
- A good test is not redundant
- “should be “best of breed”
- “neither too simple nor too complex”

Debugging

- Debugging occurs as a consequence of successful testing
- Debugging can be very challenging because the visible problem (the error you see) and the actual cause inside the code may not be directly connected or easy to find.(Shamsul Alamin Pharma)

Debugging Process

- Debugging begins with the execution of the **error generating test case** ...
- Attempts to match the symptom with probable causes ... thereby leading to error correction
- Debugging will always have one of two outcomes
 - Cause will be found and corrected
 - Cause will not be found

- Debugging involves more psychological effort than just following steps.
- However, a few characteristics of bugs provide some clues:
 - The symptom (visible problem) might appear far away from where the actual cause is in the program. This is especially true when components are highly coupled, making the problem harder to find.
 - The symptom may disappear when another error is corrected
 - The symptom may actually be caused by non errors (round-off problems)
 - It may be very difficult to reproduce the input conditions (in real-time situation)
 - The symptom may be intermittent (in embedded systems)

Psychological Considerations

- During debugging, errors can be anything from a small annoyance to a serious problem.
- As the consequence of an error increase, the amount of pressure to find the cause also increases.
- Often, pressure forces a software developer to fix one more error while at the same time introducing two more
- Unfortunately, some research suggests that the ability to debug well is partly a natural skill—some people are naturally better at it than others.
- Still, if somebody follows a good debugging strategy, they can show very good performance...

Debugging Strategies

- Debugging has one very straightforward objective: to find and correct the cause of the error
- Debugging strategies
 - Brute force
 - Backtracking
 - Cause elimination
- Automated debugging: Integrated Development Environments (IDEs) provide a way to capture some of the language specific predetermined errors without requiring compilation.
- The people factor: any discussion of debugging approaches and tools is incomplete without mention of a powerful ally—other people! A fresh pair of eyes, not tired or frustrated, can often find problems quickly. A good rule for debugging is: “When all else fails, get help!”

Correcting the Error

- Once a bug has been found, it must be corrected. But, as we have already noted, the correction of a bug can introduce other errors and therefore do more harm than good.
- Hence, after correcting the error, we need to go for Regression Testing...

